M2 - Architecture et Programmation d'accélérateurs Matériels.

(APM 2025-2026)

Controle Continu 1

Durée : 3 heures

julien.jaeger@cea.fr

Toutes les réponses doivent être justifiées.
La qualité de la rédaction est prise en compte.
All answers must be explained.

I Questions de cours

Q.1: Que fait la fonction *cudaMallocHost* ? Quelles sont les différences avec la fonction *cudaMalloc* ? et la fonction *malloc* ?
What does the cudaMallocHost function do ? What are the differences with the cudaMalloc function ? and malloc function ?

Q.2: Que signifie faire un accés coalescé à la mémoire ?
What does it mean to do a "coalesced access" to the memory ?

Q.3: Exprimer le calcul de l'indice global d'un thread CUDA en considérant que les blocs sont définis en 3D avec des tailles de BLOCK_X, BLOCK_Y et BLOCK_Z, et que la grille est définié en 2D avec des tailles de GRID_X, et GRID_Y.
Express the calculation of the global index of a CUDA thread considering that the blocks are defined in 3D with sizes BLOCK_X, BLOCK_Y and BLOCK_Z, and that the grid is defined in 2D with sizes GRID_X and GRID_Y.

Q.4: Lequel de ces mots clés n'est pas un attribut de données dans l'API CUDA.

1. ___device___
2. ___shared___
3. ___local___
4. ___constant___

Which one of the above key words are not data attributes in the CUDA API.

Q.5: Quel est l'équivalent en OpenACC du pragma OpenMP #pragma omp target data map(to :a[0 :8]) ?
What is the OpenACC equivalent of the OpenMP #pragma omp target data map(to :a[0 :8]) ?

II Succession de kernels

Nous considérons onze kernels, s'exécutant et se succédant sur trois streams différents du même GPU, selon le graphe en Figure 1. Les flèches entre les kernels représentent les dépendances de données (e.g., le kernel A produit des données utilisées par les kernels C et D).

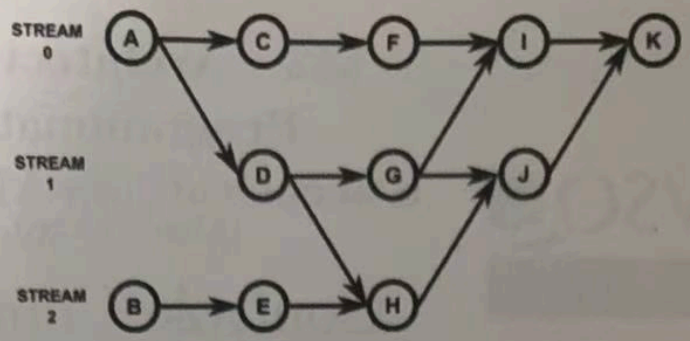

FIGURE 1 – Succession de kernels.

We consider eleven kernels running on three streams of one GPU, as shown in Figure 1. Arrows between kernels are data dependencies (e.g., kernel A produces data used by kernels C and D).

Q.6: Donner le code des appels de kernels avec les synchronisations minimales nécessaires pour avoir un comportement sûr du programme, et obtenir le bon résultat.
Write the code with the kernel calls and the minimum required synchronisations to ensure the safety of the program and produce the correct result.

III Safe/Unsafe

Dites si les sections de codes CUDA suivantes risquent de provoquer des erreurs ou des comportements non-déterministes lors de leurs exécutions.
Indicate whether the following CUDA code sections are likely to cause errors or non-deterministic behavior during their execution.

Q.7: code :

```
cudaStream_t stream[1];
cudaStreamCreate(&stream[0]);
int *htab, *dtab, *tmp;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*size);
cudaMallocHost((void**)&tmp, size);
cudaMallocHost((void**)&htab, size);
memset(htab, 0, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice, stream[0]);
kernel<<grid, block, 0, stream[0]>>(dtab, tmp, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost,stream[0]);
cudaStreamSynchronize(stream[0]); for(i=0; i<N; i++)
    printf("%d", htab[i]);
```

Q.8: code :

```
cudaStream_t stream[3];
cudaStreamCreate(&stream[0]); cudaStreamCreate(&stream[1]);
cudaStreamCreate(&stream[2]);
int *htab, *dtab;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 6*size);
cudaMallocHost((void**)&htab, 7*size);
memset(htab, 0, 7*size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + size, htab + size, 2*size, cudaMemcpyHostToDevice, stream[1]);
cudaMemcpyAsync(dtab + 3*size, htab + 3*size, 3*size, cudaMemcpyHostToDevice, stream[2]);
```

```
kernel<<<grid0, block0, 0, stream[0]>>>(dtab, N);
kernel<<<grid1, block1, 0, stream[1]>>>(dtab + size, 2*N);
kernel<<<grid2, block2, 0, stream[2]>>>(dtab + 3*size, 3*N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]); cudaMemcpyAsync(htab + size, dtab + size, 2*size,
cudaMemcpyDeviceToHost, stream[1]);
cudaStreamSynchronize(stream[1]); cudaMemcpyAsync(htab + 3*size, dtab + 3*size, 3*size,
cudaMemcpyDeviceToHost, stream[2]);
cudaStreamSynchronize(stream[2]); for(i=0; i<7*N; i++)
    printf("%d", htab[i]);
```

Q.9: code :

```
int *htab, *dtab;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*size);
cudaMallocHost((void**)&htab, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice);
kernel<<<grid, block>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost);
for(i=0; i<N-1; i++)
    htab[i] = htab[i+1];
htab[N-1] = 0;
```

Q.10: code :

```
cudaStream_t stream[2];
cudaStreamCreate(&stream[0]); cudaStreamCreate(&stream[1]);
int *htab, *dtab;
int sz_in_bytes = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*sz_in_bytes);
cudaMallocHost((void**)&htab, 2*sz_in_bytes);
memset(htab, 0, 2*sz_in_bytes);
cudaMemcpyAsync(dtab, htab, sz_in_bytes, cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + sz_in_bytes, htab + sz_in_bytes, sz_in_bytes, cudaMemcpyHostToDevice,
stream[1]);
kernel<<<grid, block, 0, stream[0]>>>(dtab + sz_in_bytes, N);
kernel<<<grid, block, 0, stream[1]>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, sz_in_bytes, cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]);
cudaMemcpyAsync(htab + sz_in_bytes, dtab + sz_in_bytes, sz_in_bytes, cudaMemcpyDeviceToHost,
stream[1]);
cudaStreamSynchronize(stream[1]);
for(i=0; i<2*N; i++)
    printf("%d", htab[i]);
```

IV Réduction somme en CUDA

Une réduction somme consiste à additionner toutes les valeurs d'une tableau. Une écriture séquentielle d'une réduction pourrait être la suivante :
A reduce-sum adds all elements in an array. A sequential version of such a computation could be :

```
float sum = 0;
for (int i = 0; i < ntot; i++)
        sum += tab[i];
```

On désire écrire une implémentation CUDA de la réduction somme (*tab* se trouve sur le GPU). Pour implémenter la réduction, nous proposons d'opérer en deux étapes :

1. Une première réduction dans chaque bloc. On obtient ainsi à la fin un tableau dimensionné au nombre de blocs et dont les valeurs sont les sommes partielles de chaque bloc.
2. Une seconde réduction sur les sommes partielles. On obtient ainsi la somme totale des éléments du tableau.

We want to provide a CUDA version of the sum-reduce, with tab amready in the GPU memory. To implement this sum-recuce, we suggest to do the following :

1. *A first reduction in each block. At the end, we have an array with a size equal to the number of blocks, which each cell containing the partial reduction of each block.*
2. *A second reduction operating on this array to reduce all the partial sum in a unique variable.*

Q.11: Implémenter le kernel `reduce_kernel(float *in, float *out)` permettant de faire les sommes partielles par bloc. *in* est le tableau de valeurs à réduire dimensionné au nombre total de threads dans la grille, et *out* le tableau de valeurs réduites par bloc, dimensionné au nombre de bloc.
Donner les deux invocations successives de cette fonction permettant de faire d'abord la somme partielle par bloc, puis la somme totale à partir des sommes partielles.
Pour simplifier, nous considérons que le nombre de blocs et de threads par blocs sont des puissances de 2, et que la taille du tableau initiale est égale au nombre de threads.

Implement the `reduce_kernel(float *in, float *out)` *kernel realizing the partial reduction per block.* `in` *is the input array with the values to reduce and with a size equivalent to the total number of threads, and* `out` *the array hosting the resulting partial reductions per block.*
Also detail the two invocations of this kernel function, first realizing the partial sums for each block, and second realizing the total reduce-sum from the partial sums.
To simplify, we consider the number of blocks and of threads per blocks are power of 2, and the initial input array as a size equal to the total number of launched threads.